Práctica N°3: Uso de base de datos FireBase en ESP32 con comunicación Wi-Fi

Índice

1. Objetivos **2**
- 1.1. Objetivo general . . . 2
- 1.2. Objetivos específicos . . . 2

2. Configuración de la base de datos Firebase **2**
- 2.1. Creación del proyecto en Firebase . . . 2
- 2.2. Activación de la base de datos en tiempo real . . . 3
- 2.3. Obtención de la URL del proyecto . . . 4
- 2.4. Configuración de autenticación por correo y contraseña . . . 5
- 2.5. Verificación de la estructura de datos . . . 7

3. Explicación del código **8**
- 3.1. Código completo . . . 8
- 3.2. Explicación por secciones . . . 13
  - 3.2.1. Encabezado e `imports` . . . 13
  - 3.2.2. Configuración (credenciales y parámetros) . . . 13
  - 3.2.3. Variables internas globales . . . 14
  - 3.2.4. `connect_wifi()`: conectar la placa a la red . . . 14
  - 3.2.5. `firebase_auth()`: autenticación contra Firebase . . . 15
  - 3.2.6. `read_firebase_data()`: lectura del nodo remoto . . . 16
  - 3.2.7. Función `main()` — flujo principal . . . 17
  - 3.2.8. Guardia de ejecución . . . 19

4. Resultados **19**
- 4.1. Salida esperada . . . 19
- 4.2. Interpretación línea por línea . . . 20

1. Objetivos

1.1. Objetivo general

Implementar y comprender el proceso completo de conexión entre una placa ESP32 y la base de datos en tiempo real de Firebase utilizando MicroPython, desde la configuración inicial del entorno hasta la ejecución del código de lectura y autenticación.

1.2. Objetivos específicos

- Configurar la interfaz de desarrollo web y los servicios necesarios en Firebase para la comunicación con el ESP32.
- Establecer la conexión WiFi del microcontrolador y validar la autenticación mediante el uso del servicio `Firebase Auth REST API`.
- Comprender la estructura y funcionamiento del código en MicroPython que permite leer datos del nodo remoto de la base de datos.
- Analizar los mecanismos de reautenticación, manejo de errores y mantenimiento de conexión dentro del bucle principal del programa.
- Verificar el correcto intercambio de datos entre la placa y Firebase mediante la consola UART del entorno Thonny.

2. Configuración de la base de datos Firebase

Antes de ejecutar el programa en el ESP32, es necesario crear e integrar un proyecto en **Firebase Realtime Database**. Esta configuración permitirá almacenar y acceder a los datos que el microcontrolador leerá mediante el servicio REST. A continuación, se detallan los pasos a seguir desde la consola web de Firebase.

2.1. Creación del proyecto en Firebase

1. Acceder a la página oficial de `https://firebase.google.com` e iniciar sesión con una cuenta de Google.
2. En el panel principal, seleccionar **"Ir a la consola"** y luego hacer clic en **"Agregar proyecto"**.

3. Asignar un nombre al proyecto, por ejemplo: `maestria-iiot-rppico`, y desactivar la opción de Google Analytics (no es necesaria para esta práctica).

4. Presionar **"Crear proyecto"** y esperar a que se complete la inicialización.

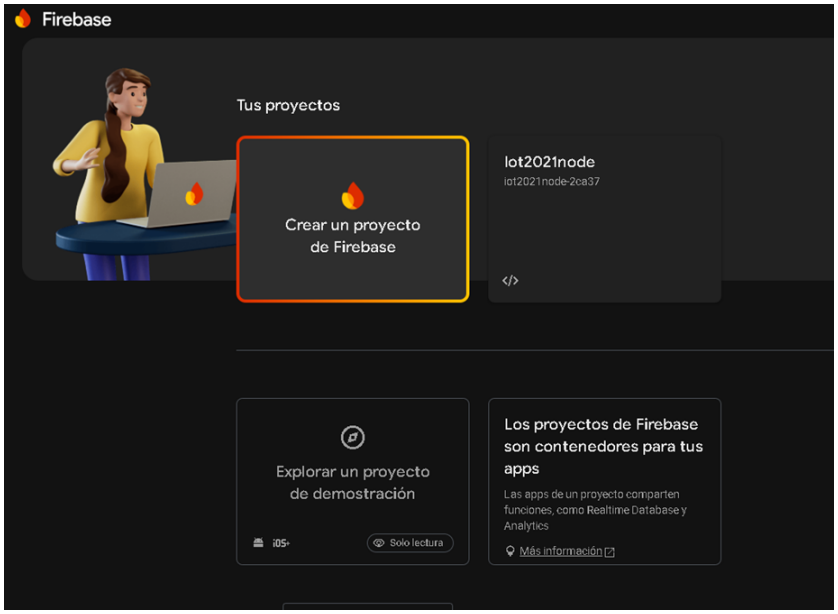


Figura 1: Creación del nuevo proyecto en Firebase.

Este paso establece el entorno donde residirán los datos leídos por el ESP32. Cada proyecto de Firebase contiene servicios independientes, por lo que mantener un nombre descriptivo ayuda a identificar su propósito.

2.2. Activación de la base de datos en tiempo real

1. En el menú lateral del proyecto, seleccionar la opción **"Base de datos"**.

2. Elegir **"Base de datos en tiempo real"** y luego hacer clic en **"Crear base de datos"**.

3. Seleccionar la ubicación del servidor (por ejemplo, `us-central1`) y escoger el modo **"Prueba"** para permitir temporalmente el acceso sin reglas estrictas de seguridad.

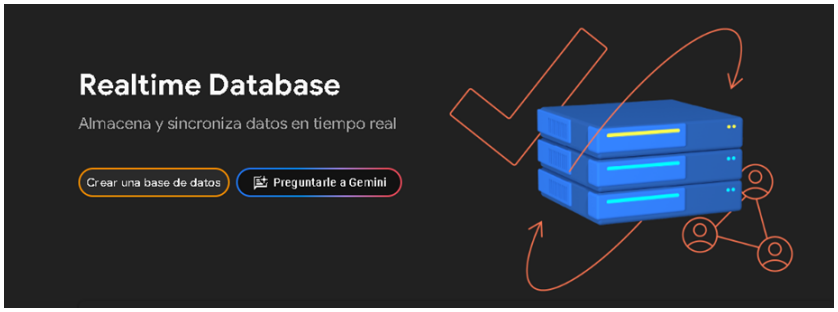


Figura 2: Inicialización del servicio Realtime Database.



Figura 3: Inicialización del servicio Realtime Database.

El modo “Prueba” facilita la depuración inicial permitiendo operaciones de lectura y escritura sin autenticación. Sin embargo, en entornos productivos debe reemplazarse por reglas de acceso controlado mediante tokens de autenticación.

2.3. Obtención de la URL del proyecto

Al crear la base de datos, Firebase genera una URL única con el formato:

`https://<nombre-del-proyecto>.firebaseio.com/`

Esta dirección debe copiarse e insertarse en el archivo `main_firebase.py` dentro de la variable `FIREBASE_URL`. Es el punto de enlace que el ESP32 usará para consultar los datos del nodo remoto.

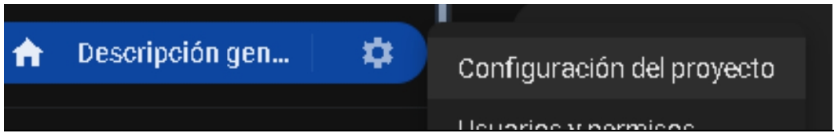


Figura 4: Obtención de la URL base de la Realtime Database.

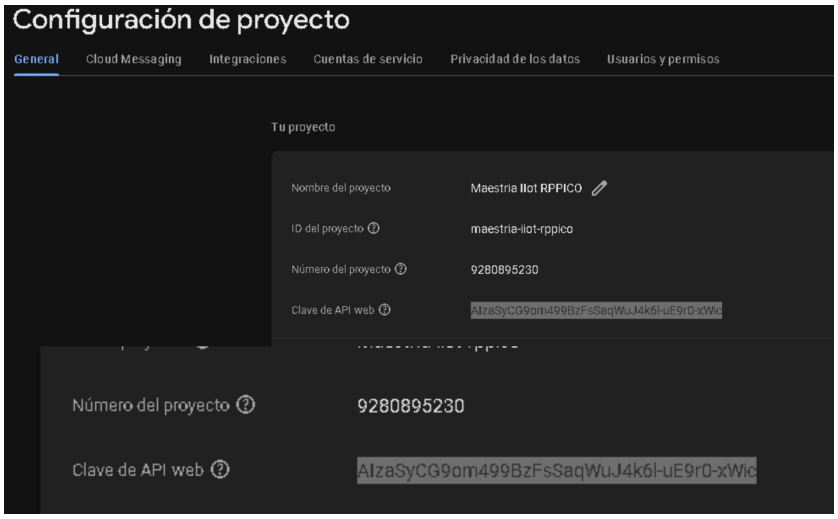


Figura 5: Obtención de la URL base de la Realtime Database.

2.4. Configuración de autenticación por correo y contraseña

1. En el panel del proyecto, acceder al apartado **"Authentication"**.
2. Hacer clic en **"Empezar"** y seleccionar el método **Correo electrónico / contraseña**.
3. Activar este método y guardar los cambios.
4. En la pestaña **Usuarios**, crear un nuevo usuario con correo y contraseña, por ejemplo:
   - Correo: `paulchasi@hotmail.com`
   - Contraseña: `Paul2179`



Figura 6: Habilitación del método de autenticación por correo y contraseña.

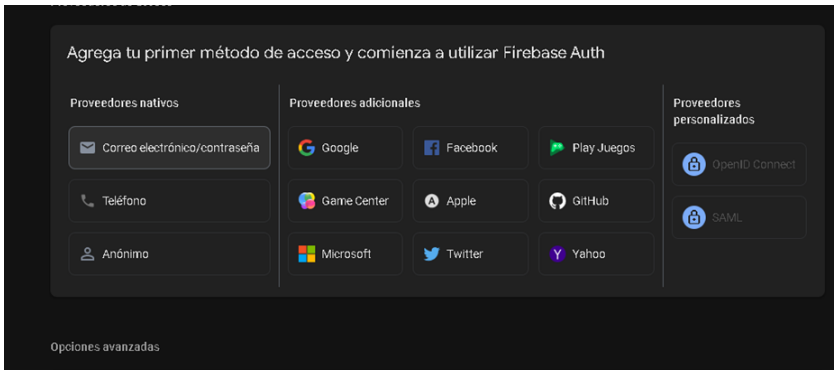


Figura 7: Habilitación del método de autenticación por correo y contraseña.



Figura 8: Habilitación del método de autenticación por correo y contraseña.

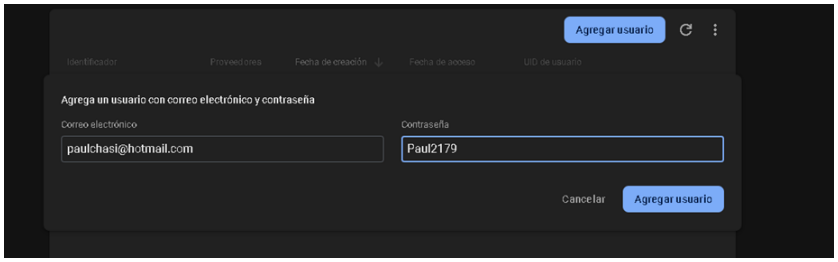


Figura 9: Habilitación del método de autenticación por correo y contraseña.

Esta credencial es la que permitirá que el ESP32 se autentique ante Firebase mediante la función `firebase_auth()` en el código. El uso de tokens evita exponer directamente las credenciales en las solicitudes de lectura y escritura.

2.5. Verificación de la estructura de datos

Dentro del editor de la base de datos, se puede crear manualmente un nodo de prueba con el nombre `/random` y valores de ejemplo:

```
{
  "data": 25,
  "temperatura": 32
}
```



Figura 10: Creación del nodo /random para pruebas de lectura.

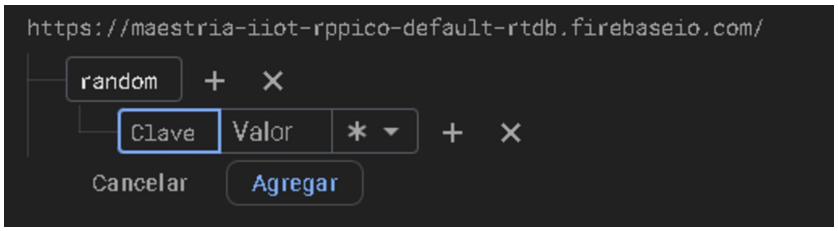


Figura 11: Creación del nodo /random para pruebas de lectura.



Figura 12: Creación del nodo /random para pruebas de lectura.



Figura 13: Creación del nodo /random para pruebas de lectura.

Este nodo será el destino de las consultas REST que el microcontrolador realizará periódicamente. Si el código funciona correctamente, los valores almacenados aquí se mostrarán en la consola UART del entorno Thonny.

3. Explicación del código

A continuación se muestra el archivo completo `main_firebase.py` que se utilizará en la práctica. Después del código, se explica cada sección.

3.1. Código completo

```
# -*- coding: utf-8 -*-
# main.py ESP32 (MicroPython)
# Guarda este archivo directamente en la placa usando Thonny.

import network
import urequests as requests
import time
import gc

# ========================
# CONFIG (credenciales)
# ========================
WIFI_SSID = 'TU_SSID'
WIFI_PASSWORD = 'TU_PASSWORD'

FIREBASE_URL =
    'https://maestria-iiot-rppico-default-rtdb.firebaseio.com/'
FIREBASE_EMAIL = 'usuario@example.com'
FIREBASE_PASSWORD = '********'
FIREBASE_API_KEY = 'AIzaSy***************'
```

```
20
21 # Intervalo de lectura en segundos (ajusta aqui)
22 POLL_INTERVAL = 3 # lectura cada 3 segundos
23
24 # =======================
25 # Variables internas
26 # =======================
27 id_token = None
28 wlan = None
29
30 # =======================
31 # Funciones
32 # =======================
33 def connect_wifi(timeout=15):
34     #Conecta el ESP32 a la red WiFi. Lanza RuntimeError si falla.
35     global wlan
36     wlan = network.WLAN(network.STA_IF)
37     wlan.active(True)
38     if wlan.isconnected():
39         print(" WiFi ya conectado. IP:", wlan.ifconfig()[0])
40         return
41
42     print(" Activando interfaz WiFi...")
43     wlan.connect(WIFI_SSID, WIFI_PASSWORD)
44     start = time.time()
45     while not wlan.isconnected():
46         if time.time() - start > timeout:
47             raise RuntimeError(" Timeout: no se pudo conectar a WiFi")
48         print(".", end="")
49         time.sleep(1)
50     print("\n WiFi conectado. IP:", wlan.ifconfig()[0])
51
52 def firebase_auth():
53     """Autentica contra Firebase Auth REST y actualiza id_token
           global."""
54     global id_token
55     auth_url ="https://identitytoolkit.googleapis.com/
56     v1/accounts:signInWithPassword?key={}".format(FIREBASE_API_KEY)
57     auth_data = {
58         "email": FIREBASE_EMAIL,
59         "password": FIREBASE_PASSWORD,
60         "returnSecureToken": True
61     }
```

```
    try:
        print("Autenticando en Firebase...")
        resp = requests.post(auth_url, json=auth_data)
        if resp.status_code == 200:
            j = resp.json()
            id_token = j.get("idToken")
            if not id_token:
                print(" No se recibio idToken en la respuesta.")
                return False
            print(" Autenticacion exitosa.")
            return True
        else:
            print("Error auth (HTTP {}): {}".format(resp.status_code,
                resp.text))
            return False
    except Exception as e:
        print(" Excepcion durante auth:", e)
        return False
    finally:
        try:
            resp.close()
        except:
            pass
        gc.collect()

def read_firebase_data():
    """Lee el nodo /random.json con el idToken actual."""
    if not id_token:
        print(" No hay id_token - autenticate primero.")
        return None
    url = "{}random.json?auth={}".format(FIREBASE_URL, id_token)
    try:
        resp = requests.get(url)
        if resp.status_code == 200:
            data = resp.json() or {}
            # Impresion formateada
            print("\n" + "="*40)
            print(" DATOS DE FIREBASE (nodo /random)")
            print("="*40)
            print(" data    :", data.get("data", "N/D"))
            print(" temperatura:",
                "{}C".format(data.get("temperatura", "N/D")))
            print("="*40)
```

```
103             return data
104         else:
105             print(" Error lectura (HTTP {}):
                    {}".format(resp.status_code, resp.text))
106             # Si token expiro, Firebase devuelve 401 o info similar
                    -> retornar None para re-auth
107             return None
108     except Exception as e:
109         print(" Excepcion en lectura:", e)
110         return None
111     finally:
112         try:
113             resp.close()
114         except:
115             pass
116         gc.collect()
117 
118 def main():
119     global id_token
120     try:
121         connect_wifi()
122     except Exception as e:
123         print(" No se pudo iniciar WiFi:", e)
124         return
125 
126     # Intentos de autenticacion (reintentar 2 veces)
127     for attempt in range(3):
128         if firebase_auth():
129             break
130         print(" Reintentando auth... ({}/{})".format(attempt+1, 3))
131         time.sleep(2)
132     else:
133         print(" No se pudo autenticar en Firebase - abortando.")
134         return
135 
136     print("\n Iniciando bucle de monitoreo (CTRL+C para detener)...")
137     last_auth_time = time.time()
138     AUTH_EXPIRY_SECONDS = 50 * 60 # reauth cada 50 min (ajustable)
139 
140     try:
141         while True:
142             data = read_firebase_data()
143             if data is None:
```

```
144                 # Posible token expirado o error puntual: intentar
                        re-autenticar una vez
145                 print(" Intentando re-autenticacion...")
146                 if not firebase_auth():
147                     print(" Re-auth fallida. Esperando y reintentando
                            mas tarde.")
148                     time.sleep(5)
149                 else:
150                     print(" Re-auth exitosa.")
151             # Reautenticar periodicamente para evitar tokens caducados
152             if time.time() - last_auth_time > AUTH_EXPIRY_SECONDS:
153                 print(" Reautenticando por tiempo transcurrido...")
154                 if firebase_auth():
155                     last_auth_time = time.time()
156             time.sleep(POLL_INTERVAL)
157     except KeyboardInterrupt:
158         print("\n Bucle detenido por usuario.")
159     except Exception as e:
160         print(" Error inesperado en main loop:", e)
161     finally:
162         try:
163             if wlan:
164                 wlan.disconnect()
165                 wlan.active(False)
166         except:
167             pass
168         gc.collect()
169         print(" Fin.")
170
171 # ======================
172 # Ejecutar
173 # ======================
174 if __name__ == "__main__":
175     main()
```

Listing 1: main$_f irebase.py^-cdigocompleto(MicroPython, ESP32)$

3.2. Explicación por secciones

3.2.1. Encabezado e `imports`

```
# -*- coding: utf-8 -*-
# main.py ESP32 (MicroPython)
# Guarda este archivo directamente en la placa usando Thonny.

import network
import urequests as requests
import time
import gc
```

Listing 2: Imports y cabecera

Primero se importan las bibliotecas necesarias: `network` para gestionar la interfaz WiFi, `urequests` (micro versión de requests) para realizar peticiones HTTP, `time` para temporización y `gc` para invocar el recolector de basura.

Se debe considera que en MicroPython el uso de memoria es limitado; por eso el código llama a `gc.collect()` en bloques `finally`. Cada petición HTTP y cada objeto JSON consume RAM.

3.2.2. Configuración (credenciales y parámetros)

```
# ========================
# CONFIG (credenciales)
# ========================
WIFI_SSID = 'TU_SSID'
WIFI_PASSWORD = 'TU_PASSWORD'

FIREBASE_EMAIL = 'usuario@example.com'
FIREBASE_PASSWORD = '********'
FIREBASE_API_KEY = 'AIzaSy***************'
FIREBASE_URL =
    'https://maestria-iiot-rppico-default-rtdb.firebaseio.com/'

# Intervalo de lectura en segundos (ajusta aqui)
POLL_INTERVAL = 3 # lectura cada 3 segundos
```

Listing 3: Variables de configuración

Se definen los parámetros necesarios para la práctica: credenciales WiFi, URL base de Firebase, API key y credenciales de autenticación (correo/clave), además del intervalo de muestreo.

- **Seguridad:** no es buena práctica dejar credenciales en texto plano. En un entorno real conviene:
  - mover credenciales a un archivo `secrets.py` (no subirlo al repositorio), o
  - solicitar credenciales por consola/puerto serie, o
  - usar variables de entorno (si el entorno lo permite).
- **POLL_INTERVAL:** escoger un intervalo mayor reduce consumo de red y CPU; en baterías conviene aumentar este valor.

3.2.3. Variables internas globales

```
# ======================
# Variables internas
# ======================
id_token = None
wlan = None
```

Listing 4: Variables internas

Se definen las reservas globales para almacenar el `id_token` de Firebase (token JWT devuelto por el servicio Auth) y el objeto `wlan` de la interfaz de red. El uso de variables globales facilita el diseño pero exige cuidado con concurrencia y estado; aquí el programa es secuencial, por lo que es aceptable.

3.2.4. `connect_wifi()`: conectar la placa a la red

```
def connect_wifi(timeout=15):
    """Conecta el ESP32 a la red WiFi. Lanza RuntimeError si
       falla."""
    global wlan
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if wlan.isconnected():
        print(" WiFi ya conectado. IP:", wlan.ifconfig()[0])
        return

    print(" Activando interfaz WiFi...")
    wlan.connect(WIFI_SSID, WIFI_PASSWORD)
    start = time.time()
    while not wlan.isconnected():
        if time.time() - start > timeout:
```

```
            raise RuntimeError(" Timeout: no se pudo conectar a WiFi")
        print(".", end="")
        time.sleep(1)
    print("\n WiFi conectado. IP:", wlan.ifconfig()[0])
```

Listing 5: Función connect_wifi

Mediante la interfaz WiFi en modo station, inicia la conexión y espera hasta que la conexión se establezca o ocurra un timeout.

- **Timeout:** evita bloqueos indefinidos; el valor por defecto es 15 s.
- **Mensajes por UART:** se usan `print()` para informar al usuario — en la práctica esto se verá en la consola de Thonny.

3.2.5. `firebase_auth()`: autenticación contra Firebase

```
def firebase_auth():
    """Autentica contra Firebase Auth REST y actualiza id_token
        global."""
    global id_token
    auth_url = "https://identitytoolkit.googleapis.com/
    v1/accounts:signInWithPassword?key={}".format(FIREBASE_API_KEY)
    auth_data = {
        "email": FIREBASE_EMAIL,
        "password": FIREBASE_PASSWORD,
        "returnSecureToken": True
    }
    try:
        print(" Autenticando en Firebase...")
        resp = requests.post(auth_url, json=auth_data)
        if resp.status_code == 200:
            j = resp.json()
            id_token = j.get("idToken")
            if not id_token:
                print(" No se recibio idToken en la respuesta.")
                return False
            print(" Autenticacion exitosa.")
            return True
        else:
            print(" Error auth (HTTP {}):
                {}".format(resp.status_code, resp.text))
            return False
    except Exception as e:
```

```
        print(" Excepcion durante auth:", e)
        return False
    finally:
        try:
            resp.close()
        except:
            pass
        gc.collect()
```

Listing 6: Función firebase_auth

Para establecer conexión se realiza una petición POST al endpoint de Firebase Auth para obtener un `idToken`. Si la respuesta es correcta guarda el token en la variable global `id_token`.

Detalles técnicos:

- El endpoint utilizado es el provisto por Google Identity Toolkit; la `API_KEY` forma parte del URL.
- El token que devuelve Firebase tiene caducidad; por eso el programa implementa re-autenticación periódica.
- Se captura cualquier excepción para evitar que el dispositivo colapse.

3.2.6. `read_firebase_data()`: lectura del nodo remoto

```
def read_firebase_data():
    """Lee el nodo /random.json con el idToken actual."""
    if not id_token:
        print(" No hay id_token - autenticate primero.")
        return None
    url = "{}random.json?auth={}".format(FIREBASE_URL, id_token)
    try:
        resp = requests.get(url)
        if resp.status_code == 200:
            data = resp.json() or {}
            # Impresion formateada
            print("\n" + "="*40)
            print(" DATOS DE FIREBASE (nodo /random)")
            print("="*40)
            print(" data    :", data.get("data", "N/D"))
            print(" temperatura:",
                "{}C".format(data.get("temperatura", "N/D")))
            print("="*40)
```

```
            return data
        else:
            print(" Error lectura (HTTP {}):
                {}".format(resp.status_code, resp.text))
            # Si token expiro, Firebase devuelve 401 o info similar
                -> retornar None para re-auth
            return None
    except Exception as e:
        print(" Excepcion en lectura:", e)
        return None
    finally:
        try:
            resp.close()
        except:
            pass
        gc.collect()
```

Listing 7: Función read_firebase_data

ESte bloque construye la URL para el nodo `/random.json` (se añade `?auth=<idToken>`) y hace una petición GET. Si tiene éxito muestra por UART los campos `data` y `temperatura`.

- El uso de `?auth=` con el token es el modo estándar de autenticación para Realtime DB en peticiones REST.
- Si la lectura falla (por token expirado u otros), la función retorna `None` para que el flujo principal intente re-autenticar.
- **Robustez:** conviene manejar códigos HTTP específicos (401, 403) de forma diferenciada.

3.2.7. Función `main()` — flujo principal

```
def main():
    global id_token
    try:
        connect_wifi()
    except Exception as e:
        print(" No se pudo iniciar WiFi:", e)
        return

    # Intentos de autenticacion (reintentar 2 veces)
    for attempt in range(3):
```

```
        if firebase_auth():
            break
        print(" Reintentando auth... ({}/{})".format(attempt+1, 3))
        time.sleep(2)
    else:
        print(" No se pudo autenticar en Firebase  abortando.")
        return

    print("\n Iniciando bucle de monitoreo (CTRL+C para detener)...")
    last_auth_time = time.time()
    AUTH_EXPIRY_SECONDS = 50 * 60 # reauth cada 50 min (ajustable)

    try:
        while True:
            data = read_firebase_data()
            if data is None:
                # Posible token expirado o error puntual: intentar
                    re-autenticar una vez
                print(" Intentando re-autenticacion...")
                if not firebase_auth():
                    print(" Re-auth fallida. Esperando y reintentando
                        mas tarde.")
                    time.sleep(5)
                else:
                    print(" Re-auth exitosa.")
            # Reautenticar periodicamente para evitar tokens caducados
            if time.time() - last_auth_time > AUTH_EXPIRY_SECONDS:
                print(" Reautenticando por tiempo transcurrido...")
                if firebase_auth():
                    last_auth_time = time.time()
            time.sleep(POLL_INTERVAL)
    except KeyboardInterrupt:
        print("\n Bucle detenido por usuario.")
    except Exception as e:
        print(" Error inesperado en main loop:", e)
    finally:
        try:
            if wlan:
                wlan.disconnect()
                wlan.active(False)
        except:
            pass
        gc.collect()
```

```
52         print(" Fin.")
```

Listing 8: Función main y bucle principal

Aquí se organiza la ejecución completa:

1. conecta a WiFi,
2. intenta autenticarse (hasta 3 intentos),
3. entra en un bucle permanente donde lee datos periódicamente,
4. re-autentica si detecta error en lectura o si ha pasado suficiente tiempo.

3.2.8. Guardia de ejecución

```
# ========================
# Ejecutar
# ========================
if __name__ == "__main__":
    main()
```

Listing 9: Ejecución condicional

Este bloque asegura que `main()` se ejecute sólo cuando el archivo se lance como script principal (útil para pruebas unitarias o para importar funciones desde otro script).

4. Resultados

4.1. Salida esperada

A continuación se muestra un extracto de la salida que el estudiante debería ver en la consola. Este bloque sirve como referencia para comparar con la propia ejecución.

```
 Activando interfaz WiFi...
.....
 WiFi conectado. IP: 192.168.1.42
 Autenticando en Firebase...
 Autenticacion exitosa.

 Iniciando bucle de monitoreo (CTRL+C para detener)...
========================================
 DATOS DE FIREBASE (nodo /random)
```

```
10 ========================================
11  data    : 25
12  temperatura: 32C
13 ========================================
14
15 ... (pausa de 3 s por POLL_INTERVAL) ...
16  Intentando re-autenticacin...
17  Re-auth exitosa.
```

Listing 10: Salida de ejemplo en la consola (Thonny)

4.2. Interpretación línea por línea

- **“ Activando interfaz WiFi...”** – El ESP32 ha activado su interfaz en modo station y comienza el intento de conexión. Si la línea no aparece, revisa conexiones o puerto serie.
- **“.....”** – puntos que indican espera mientras se obtiene IP. Si aparecen demasiados puntos y después un `Timeout`, revisar SSID/clave o cobertura WiFi.
- **“ WiFi conectado. IP: 192.168.1.42”** – Conexión exitosa; la IP puede variar según tu red.
- **“ Autenticando en Firebase...”** – Inicio del POST a Firebase Auth. Si aquí hay un fallo, el error suele ser 400/401 por credenciales/API_KEY incorrecta.
- **“ Autenticacion exitosa.”** – Se obtuvo el `idToken` y ya se puede consultar la Realtime Database.
- **Bloque de datos (=...)** – Formato de impresión legible que muestra los campos leídos del nodo `/random`. Comprueba que `data` y `temperatura` coincidan con los valores en la consola web de Firebase.
- **“... (pausa de 3 s por POLL_INTERVAL) ...”** – Indica la espera programada entre lecturas.
- **“ Intentando re-autenticación...”** / **“ Re-auth exitosa.”** – Flujo de reautenticación si el token caduca o si hubo fallos transitorios en la lectura.

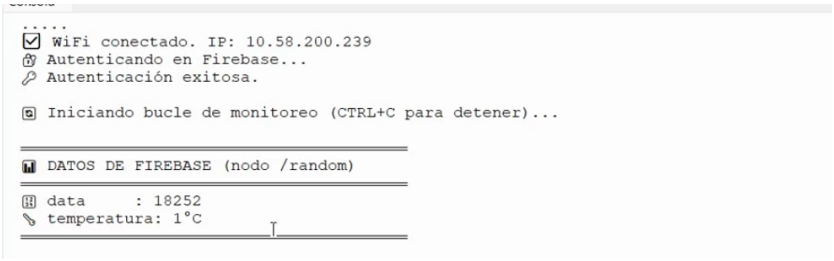


Figura 14: Salida en el monitor serial de Thonny.